Problem Set 1

Fundamentals of Numerical Programming I

Leonard Storcks

December 22, 2025

Problem 1. Short questions

(a) What is the largest number you can represent with a 2-byte **unsigned** integer (expressed in decimal)?

(b) What is the largest number you can represent with a 2-byte **signed** integer (expressed in decimal)?

(c) When running `char c = 100 * 4;` in C, -112 is stored. Why does this happen? Note a char is a single byte.

(d) What is the meaning of the machine precision ϵ?

(e) When is $a + b$ typically equal to a in limited precision floating point arithmetic?

(f) Can 0.1 be exactly represented as a (base-2) floating-point number?

Solution

Part (a)

$n = 2$ bytes are $8n = 16$ bits, with which we can encode 2^{8n} states, so as we start counting from 0, the largest number is $2^{8n} - 1 = 2^{16} - 1 = 65535$.

Part (b)

We need one bit for the sign, so generally $2^{8n-1} - 1$ is the largest number we can represent, so for $n = 2$ bytes $2^{16-1} - 1 = 32767$.

Part (c)

In `char c = 100 * 4; // range -128 to 127` the result is -112 as 400 in bits is 000110010000 where a cut-off to one byte means 10010000 so $-2^7 + 2^4 = -128 + 16 = -112$ (two's complement).

Part (d)

The smallest increment in the mantissa, in $1 + \frac{M}{2^p}$, is the **machine precision**.

Part (e)

$a + b = a$ typically for $|b| < \epsilon_{\text{mach}}|a|$, i.e. when b cannot be resolved by the mantissa of a.

Part (f)

We can only exactly represent numbers which can be written in the form $V = x \cdot 2^y$, with integers $x, y \in \mathbb{Z}$. This is not possible for 0.1 ($0.1_{10} = 1.10011[0011]\ldots_2 \cdot 2^{-4}$).

Problem 2. Basler Problem

Consider the famous series

$$\sum_{k=1}^{\infty} k^{-2} = \frac{\pi^2}{6} \tag{1}$$

known as the Basler problem. We would like to approximate this series by finitely many summands.

(a) For a given finite expression $1 + \frac{1}{2^2} + \cdots + \frac{1}{n^2}$ will it make a difference if we sum the number from small to large or from large to small on the computer at finite precision?

(b) Implement both the summation from large to small and small to large in single precision (use np.float32) and compare the results to the ground truth, $\frac{\pi^2}{6}$.

Solution

Part (a)

If we sum the terms just as the formula suggests from large to small, at some point, the small changes will not be resolved anymore - so better sum up from small to large.

Part (b)

See Code-Snippet 1.

```python
import numpy as np
import math

def basel_sum_small_to_large_f32(n):
    s = np.float32(0.0)
    for k in range(n, 0, -1):
        s += np.float32(1.0) / np.float32(k * k)
    return s

def basel_sum_large_to_small_f32(n):
    s = np.float32(0.0)
    for k in range(1, n + 1):
        s += np.float32(1.0) / np.float32(k * k)
    return s

# Ground truth (double precision)
exact = math.pi**2 / 6

for n in [10**3, 10**4, 10**5, 10**6]:
    s_sl = basel_sum_small_to_large_f32(n)
    s_ls = basel_sum_large_to_small_f32(n)

    err_sl = abs(float(s_sl) - exact)
    err_ls = abs(float(s_ls) - exact)

    print(f"n = {n:>7}")
    print(f"  small -> large (float32): {s_sl:.7f}, error = {err_sl:.3e}")
    print(f"  large -> small (float32): {s_ls:.7f}, error = {err_ls:.3e}")
    print()
```

Code-Snippet 1: Python code for approximating $\sum_{k=1}^{\infty} k^{-2}$ using n summands. Summation from large to small and from small to large in finite precision are compared.

Problem 3. Rewriting expressions to avoid cancellation

Consider the limit of $x \to 0$ of the function

$$f(x) = \frac{x + e^{-x} - 1}{x^2}, \quad x > 0 \tag{2}$$

(a) What is the analytical limit?

(b) Write a program which plots $f(x)$ for $10^{-11} < x < 10^{-1}$. You should observe problematic behavior. Below which upper limit do you observe this problematic behavior? For very small x, the function evaluates to zero in finite precision. Why is that?

(c) Provide a suitable fix.

Solution

Part (a)

Taylor expand e^{-x} around $x = 0$ or use L'Hopital's rule to find that the limit is $1/2$, e.g. with the expansion of $e^{-x} = 1 - x + \frac{x^2}{2} + \mathcal{O}(x^3)$ (with Landau notation) we have

$$f(x) = \frac{x + \left(1 - x + \frac{x^2}{2} + \mathcal{O}(x^3)\right) - 1}{x^2} = \frac{\frac{x^2}{2} + \mathcal{O}(x^3)}{x^2} = \frac{1}{2} + \mathcal{O}(x) \tag{3}$$

Part (b)

The code is given in Code-Snippet 2 and the plot in Fig. 1.

Consider the exponential is calculated via the power series expansion. For very small x, the $x^2/2$ term in $1 - x + x^2/2 + \mathcal{O}(x^3)$ cannot be resolved faithfully against 1, remember $a + b$ evaluates to a typically for $|b| < \epsilon_{\text{mach}}|a|$. In double precision, this is the case for $x = 10^{-8}$

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.logspace(-1, -12, num=1000)
fx = (x + np.exp(-x) - 1) / x**2
plt.plot(x, fx)
plt.xscale('log')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Unsafe implementation')
```

Code-Snippet 2: Plotting $f(x) = \frac{x+e^{-x}-1}{x^2}$.

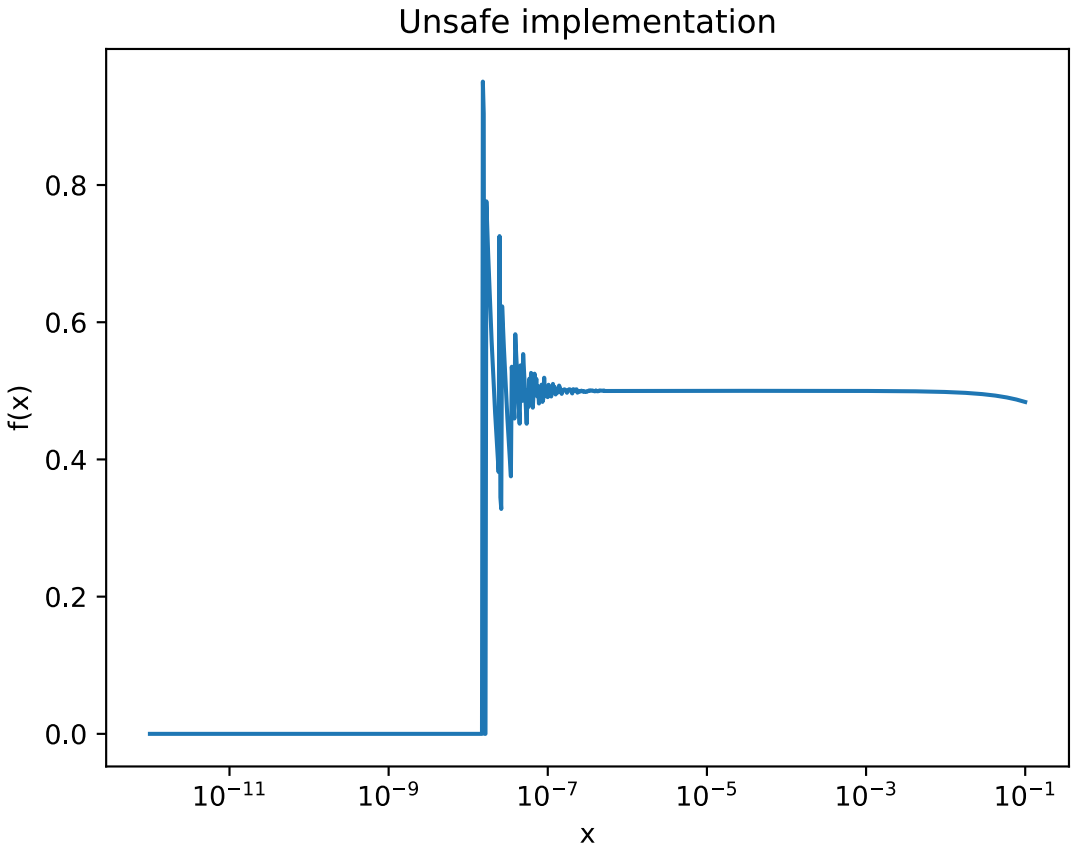

Figure 1: Plot of $f(x) = \frac{x+e^{-x}-1}{x^2}$.

and for $x = 10^{-7}$ accuracy of $x^2/2$ is already poor.

Part (c)

We define

$$f_{\text{safe}}(x) = \begin{cases} f(x), & x > 10^{-5} \\ \frac{1}{2} + \frac{1}{6}x + \frac{1}{24}x^2, & x \leq 10^{-5} \end{cases} \tag{4}$$

The code is given in Code-Snippet 3 and the plot in Fig. 2.

For general settings where $\exp -x - 1$ occurs, many numerical toolboxes like `numpy` provide a function `expm1` such that we do not have to resolve the higher order terms against 1.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.logspace(-1, -12, num=1000)
fx = (x + np.exp(-x) - 1) / x**2
plt.plot(x, fx)
plt.xscale('log')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Safe implementation')
```

Code-Snippet 3: Safe version of $f(x) = \frac{x+e^{-x}-1}{x^2}$ for finite precision.

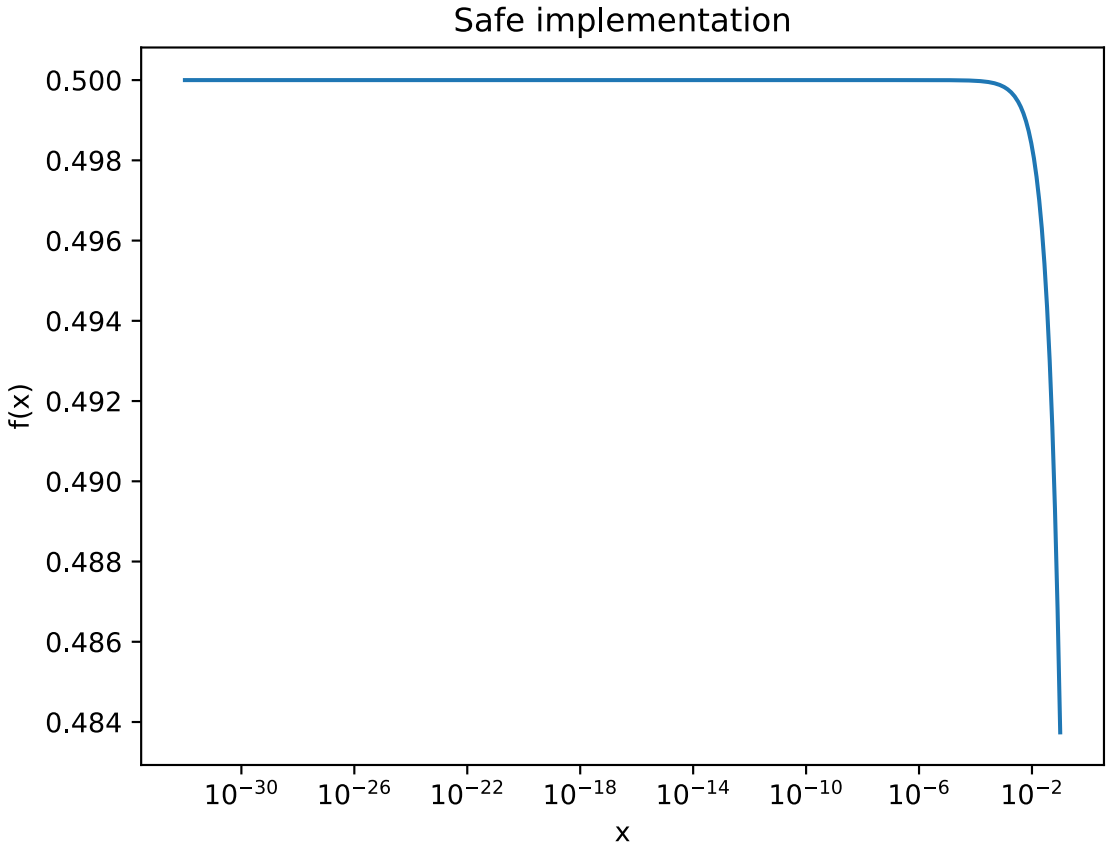

Figure 2: Plot corresponding to Code-Snippet 3